Why Architecture Choice Matters in Symbolic Regression

Anonymous Author(s)

Abstract

Symbolic regression discovers mathematical formulas from data. Some methods fix a tree of operators, assign learnable weights, and train by gradient descent. The tree's structure, which determines what operators and variables appear at each position, is chosen once and applied to every target. This paper tests whether that choice affects which targets are actually recovered. Three structures are compared, all sharing the same operator and target language but differing in how variables enter the tree; one is strictly more expressive. Across over 12,700 training runs, one structure recovers a target at 100% while another scores 0%, and the ranking reverses on a different target. Expressiveness guarantees that a solution exists in the search space, but not that gradient descent finds it: the most expressive structure fails on targets that a restricted alternative solves reliably. Switching the operator changes which targets succeed; reversing its gradient profile collapses recovery entirely. Balanced (non-chain) tree shapes are never recovered. These findings show that the optimization landscape, not expressiveness alone, determines what gradient-based symbolic regression recovers.

1 Introduction

Symbolic regression (SR) discovers mathematical formulas from data [2, 4, 11]. One family of methods parameterizes a formula as a tree of operators whose inputs are controlled by learnable weights, and recovers the target by training those weights with gradient descent [3, 14]. The tree's structure, which determines what operators and variables are available at each position, is its *architecture*: a design choice made once and applied to every target.

Prior work has compared SR methods that differ in architecture, grammar, and search simultaneously: SRBench [2] tests 14 methods on 252 problems but cannot isolate architecture effects. Gradient-based methods such as EQL [14] and eEQL [3] each use a single fixed architecture without studying how architecture interacts with targets. MetaSymNet [10] meta-learns the architecture but does not explain why fixed ones fail. Reinforcement-learning approaches [1, 9, 13] avoid fixed-architecture landscapes entirely. On the EML side, Odrzywolek [12] demonstrated SR with one architecture, and Ipek [6, 7] applied EML to hardware and battery models without interaction analysis. No prior study has varied architecture while holding the target set constant.

This paper isolates the architecture variable by using the EML (Exp-Minus-Log) operator [12], a single binary operation from which all elementary functions can be constructed. Because every node in an EML tree computes the same operation, targets differ only in tree shape and variable placement, making it possible to compare architectures that share the same target language. Three architectures are tested across more than 12,700 training runs; the results show that a target's recoverability depends on architecture, not just on whether the architecture can express it.

Table 1: Architectures at depth 3.

Name	Internal selection	Params	Variable routing
Eq. 6	softmax $\{1, x, f\}$	42	x at all nodes; y leaves only
V16	sigmoid $\{1, f\}$	38	x, y leaves only (symmetric)
Hybrid	V16 + root 4-way	42	x, y root and leaves

2 Background

The EML operator [12] computes $\mathrm{eml}(a, b) = e^a - \ln b$. Its two inputs receive asymmetric gradients: the left input is exponentially amplified ($\partial\mathrm{eml}/\partial a = e^a$) while the right is attenuated ($\partial\mathrm{eml}/\partial b = -1/b$). EML is a Sheffer operator: with only the constant 1, binary trees of EML nodes can express every elementary function (+, −, ×, /, exp, ln, trigonometric, and more) [12]. For instance, $e^x = \mathrm{eml}(x, 1)$ and $e = \mathrm{eml}(1, 1)$.

A depth-n EML tree is a perfect binary tree with $2^n - 1$ operator nodes (depth 3 gives 7 nodes with 8 leaf positions). At depth 3, five distinct shapes exist. Four are *chains*, in which one branch at each level carries the computation while the other is a leaf. Chains are classified by which EML input (Left or Right) carries the active branch:

Shape	Template	Active-branch path
LR	$\mathrm{eml}(\mathrm{eml}(1, \mathrm{eml}(\cdot, \cdot)), 1)$	Left at root, Right at depth 1
RL	$\mathrm{eml}(1, \mathrm{eml}(\mathrm{eml}(\cdot, \cdot), 1))$	Right at root, Left at depth 1
RR	$\mathrm{eml}(1, \mathrm{eml}(1, \mathrm{eml}(\cdot, \cdot)))$	Right at both levels
LL	$\mathrm{eml}(\mathrm{eml}(\mathrm{eml}(\cdot, \cdot), 1), 1)$	Left at both levels

The fifth shape is *balanced*: both root branches are full subtrees, giving $\mathrm{eml}(\mathrm{eml}(\cdot, \cdot), \mathrm{eml}(\cdot, \cdot))$. The LL chain produces double exponentiation (e^{e^a}), which overflows on the training domain and is excluded from experiments.

Three architectures are tested, summarized in Table 1. All three can represent any depth-3 EML tree with $\{1, x, y\}$ at the leaves; one is strictly more expressive. **Eq. 6** is the architecture described in Equation 6 of Odrzywolek [12]: at each internal node a softmax selects among $\{1, x, f_{\mathrm{child}}\}$, giving variable x access to every level of the tree. Because its input set at internal nodes is a strict superset of V16's, Eq. 6 is strictly more expressive. **V16** is variant 16 in the released source code[1] accompanying Odrzywolek [12]: a sigmoid at each internal node selects between $\{1, f_{\mathrm{child}}\}$, and variables x and y enter only at the leaves through a 3-way softmax, making them symmetric. **Hybrid** uses V16's sigmoid at all internal nodes except the root, where a 4-way softmax adds x and y as additional options; the name reflects its combination of V16's body with partial variable access at the root.

[1] Code: https://github.com/aodrzywolek/EML

3 Methodology

To test whether the architecture–target interaction is specific to EML or general to operators with gradient asymmetry, two additional operators are introduced:

$$\mathrm{sml}(a, b) = \sinh(a) - \arctan(b), \tag{1}$$

$$\mathrm{rml}(a, b) = \arctan(a) - \sinh(b). \tag{2}$$

SML shares EML's gradient direction: the left input receives unbounded gradients ($\partial\mathrm{sml}/\partial a = \cosh a$, which grows exponentially) while the right is bounded ($\partial\mathrm{sml}/\partial b = -1/(1 + b^2)$). RML reverses this asymmetry. Neither SML nor RML is known to be universal; target trees are evaluated numerically.

Each node's inputs are weighted combinations of the available terms, with weights controlled by learnable parameters. Training minimizes mean squared error using Adam [8] (learning rate 0.01) on a 21×21 grid over $[-3, 3]^2$ (441 points; targets with numerical overflow are filtered, leaving $n_{\text{train}} \geq 378$). A two-phase schedule is used: a *search phase* (6,000 iterations at temperature $\tau = 2.5$, keeping softmax/sigmoid weights soft) followed by a *hardening phase* (2,000 iterations; τ anneals from 2.5 to 0.01 with entropy and binarity penalties, then snaps to one-hot selections). After hardening each node selects exactly one input, producing a concrete expression tree.

Each combination of architecture, operator, tree shape, and leaf assignment is tested with 256 independent trials (64 seeds $\times$ 4 initializations) for V16 and Hybrid, or 64 trials (one initialization) for Eq. 6. A trial counts as *exact recovery* if the hardened tree matches the target formula, verified on 500 random points (RMSE $< 10^{-6}$). Replacing V16's initialization with Eq. 6's (randn $\times$ 0.1, no bias) yielded 0/64 recoveries, confirming that the architecture differences reported below are not explained by initialization.

4 Results

4.1 Architecture–Target Interaction

Table 2 shows that recovery depends on the combination of architecture and target, not on either factor alone. Eq. 6 acts as a specialist: 100% recovery (64/64) on exactly two targets and 0% (0/64) on three others. V16 is a generalist with nonzero recovery on all four chain shapes (16–99.6%) and $x \leftrightarrow y$ symmetry from its leaf-only variable design. Hybrid is an RL specialist (95–96% on RL targets, near 0% elsewhere). No single architecture dominates: Eq. 6 scores 100% where V16 scores 17%, yet V16 scores 99.6% where Eq. 6 scores 0%. Expressiveness does not explain this pattern because Eq. 6 is strictly more expressive than V16 yet fails on targets V16 recovers. The difference lies in how each architecture's variable routing interacts with the optimization landscape for a given target.

4.2 Operator Swap Reverses Shape Preferences

Switching from EML to SML (Table 3, Figure 1) inverts Eq. 6's shape preferences: its best shape under EML is LR (100%), but under SML it shifts to RL and RR (100%) while LR drops to 0% (Fisher exact $p < 10^{-15}$). Both operators amplify the left input, so the reversal traces to differences in right-side attenuation ($-1/(1 + b^2)$ for SML versus $-1/b$ for EML), which saturate differently on the $[-3, 3]^2$ domain. Despite the shape reversal, qualitative profiles persist: Eq. 6 remains a specialist, V16 a generalist (83–100% on all chains), and Hybrid an RL specialist. These profiles likely reflect structural properties of variable routing rather than artifacts of one operator.

Table 2: EML recovery rates (%, exact match).

Target	Shape	Eq. 6	V16	Hybrid
Paper (yx)	LR	**100** (64/64)	17	0.4
T1 (xy)	RL	0 (0/64)	**99.6**	**96**
T4 (xy)	RR	0 (0/64)	39.5	0
T8 (xy)	LR	1.6	16	0.4
T1_yx	RL	0	—	**95.3**
T4_yx	RR	**100** (64/64)	—	0

Table 3: SML recovery rates (%, exact match).

Target	Shape	Eq. 6	V16	Hybrid
S_LR (yx)	LR	0	84	0.8
S_LR_xy	LR	0	83	0.8
S_RL (xy)	RL	**100**	**100**	**100**
S_RL_yx	RL	0	**100**	**100**
S_RR (xy)	RR	**100**	99.6	89.8
S_RR_yx	RR	**100**	98.8	88.7

V16 dominates under SML, recovering all chain targets at 83% or above. Replacing EML with the right-amplified RML collapses all rates: Eq. 6 scores 0% on every target, V16's best drops to 2.0%, and Hybrid's RL preference vanishes. RR and balanced shapes were not tested under RML because the collapse on chain targets, which are easier to recover than balanced shapes, made further experiments uninformative.

4.3 Balanced Trees Are Universally Hard

The balanced depth-3 shape is tested with four leaf-placement variants per architecture–operator combination (Table 4). Every architecture scores 0% on every balanced variant (0/3,776 trials; Clopper–Pearson 95% upper bound 0.10%). The failure persists across 6 training configurations varying learning rate (0.01, 0.05), iteration budget (6,000, 12,000), and temperature (1.0, 2.5): 0/192. The contrast with chain shapes is stark: V16 recovers 83–100% of SML chains yet scores exactly 0% on balanced variants of the same operator. A plausible explanation is that balanced branching splits gradient signal roughly equally between the two root subtrees, preventing the concentration along a single path that drives chain recovery; gradient measurements for balanced shapes were not collected, and verifying this hypothesis is future work.

4.4 Gradient Measurements

To investigate the mechanism behind the architecture–target interaction, gradient norms at leaf parameters are measured at iteration 1000 across 10 seeds (Table 5). For Eq. 6, the ratio tracks recovery: when $x/y < 1$ (variable y receives stronger gradients), recovery is 100%; when $x/y > 1$ (y is gradient-starved), recovery is 0%. Swapping EML for SML on the same shape (LR) flips the ratio from 0.67 to 1.78, consistent with the observed preference

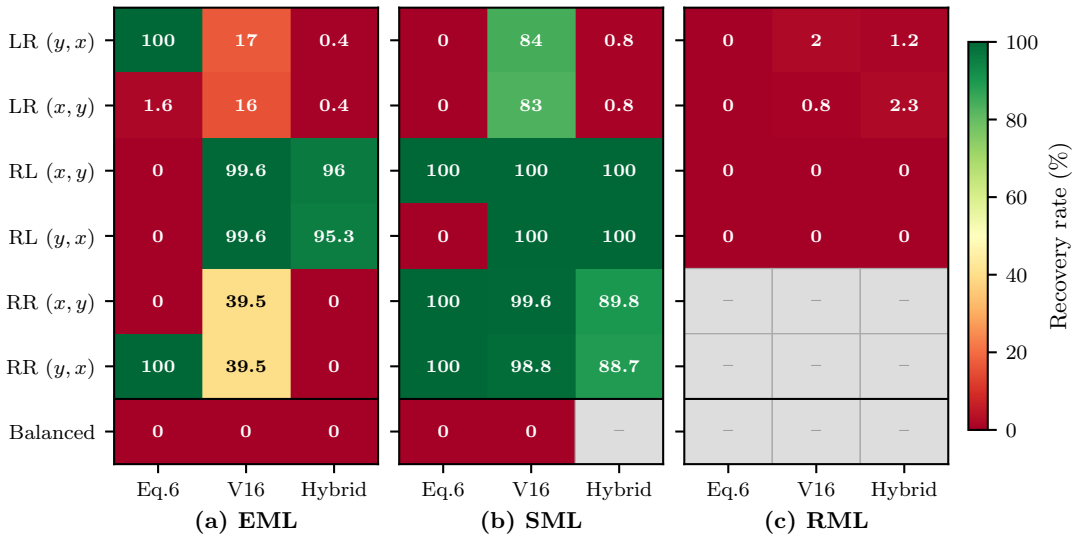

Figure 1: Recovery by architecture, shape, and operator (— = inferred by symmetry).

Table 4: Balanced recovery: 0% across all combinations.

Operator	Eq. 6	V16	Hybrid	Total
SML (4 variants)	0/256	0/1024	0/1024	0/2,304
EML (4 variants)	0/256	0/1024	—	0/1,280
HP sensitivity	—	0/192	—	0/192
Total				**0/3,776**

Table 5: Leaf gradient ratio $\|\nabla_x\|/\|\nabla_y\|$ at iteration 1000 (10-seed mean ± s.e.).

Arch.	Target	Op.	x/y ratio	Rec.
Eq. 6	LR (yx)	EML	0.67 ± 0.03	100%
Eq. 6	RL (xy)	EML	1.58 ± 0.04	0%
Eq. 6	LR (yx)	SML	1.78 ± 0.12	0%
Eq. 6	RL (xy)	SML	1.19 ± 0.11	100%
V16	LR (yx)	EML	0.77 ± 0.05	17%
V16	RL (xy)	EML	2.00 ± 0.08	99.6%

reversal. V16 shows narrower ratios across targets (0.77–2.00), consistent with its symmetric design preventing either variable from dominating the gradient landscape.

The gradient trajectory during training (Figure 2) reveals the mechanism more clearly. On the 100% EML target the ratio undergoes a brief reversal, from 1.45 at iteration 100 to 0.14 at iteration 500, then settling at 0.67 by iteration 1000; this temporary window in which y dominates correlates with convergence to the correct formula. On the 0% target the ratio remains above 1.15 at every measured iteration, and y never receives sufficient gradient signal. These measurements establish a gradient-level correlate of

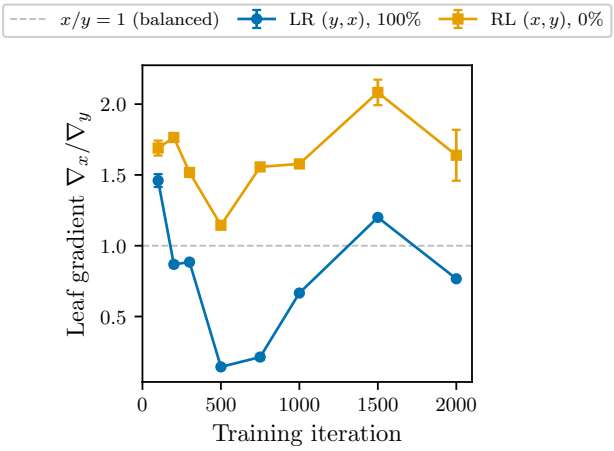

Figure 2: Gradient ratio $\|\nabla_x\|/\|\nabla_y\|$ during training (Eq. 6, 10-seed mean ± s.e.).

the architecture–target interaction; the question of causation is addressed in Section 5.1.

5 Discussion

Selecting a single architecture for all targets, as current gradient-based SR methods do [3, 12, 14], can yield 0% recovery on targets that a different architecture recovers at 100%. Neural architecture search (NAS) also studies architecture–task interaction, but NAS architectures typically differ in expressiveness [5]; here all architectures share the same target language, so recovery differences arise purely from optimization landscape geometry, a condition rarely met in benchmarks where methods differ simultaneously in grammar, search, and architecture [2].

The cross-operator results point to four interacting factors that determine recovery: variable routing, tree shape, leaf assignment, and gradient direction. Four predictions were formulated before the RML and balanced experiments. One was confirmed (V16 preferences invert under RML); three were falsified (balanced trees were predicted to vary by architecture but proved universally hard; Eq. 6 and Hybrid were predicted to reverse preferences under RML but collapsed instead). Gradient direction interacts with variable routing more strongly than a simple reordering model predicts.

5.1 Limitations

All three operators share the subtraction form $f(a) - g(b)$; whether the interaction extends to multiplicative or compositional operators is untested. The full three-architecture comparison exists only at depth 3; limited depth-4 experiments with V16 show the generalist profile collapses (RL drops from 99.6% to 0.8%), but a complete depth-4 matrix remains future work.

The controlled-expressiveness design requires all targets to lie within the EML grammar, preventing direct comparison with standard SR benchmarks (SRBench, Feynman equations) that would require exponential-depth EML trees. SML targets have narrower output range than EML targets on $[-3, 3]^2$, which may partly explain V16's higher SML rates (83–100% versus 16–99.6%). The shape-specific patterns (Eq. 6 under SML: LR 0%, RL 100%) cannot be explained by conditioning alone, since better conditioning would raise all shapes uniformly.

The gradient measurements establish correlation, not causation. A mechanistic account via loss-landscape analysis would strengthen the framework but is beyond the scope of this study.

6 Conclusion

The optimization landscape, not expressiveness alone, determines what gradient-based symbolic regression recovers. This motivates two practical directions: multi-architecture ensembles that run several structures in parallel and select the best result, and architecture-aware search that adapts variable routing to the target during optimization. The universal failure on balanced trees points to a deeper constraint that warrants direct investigation via loss-landscape analysis. Extending the interaction analysis to multiplicative operators, deeper trees, and standard SR benchmarks would test how far these findings generalize.

References

[1] Luca Biggio, Tommaso Bendinelli, Alexander Neitz, Aurelien Lucchi, and Giambattista Parascandolo. 2021. Neural Symbolic Regression that Scales. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*. 936–945.

[2] William La Cava, Patryk Orzechowski, Bogdan Burlacu, Fabrício Olivetti de França, Marco Virgolin, Ying Jin, Michael Kommenda, and Jason H. Moore. 2021. Contemporary Symbolic Regression Methods and their Relative Performance. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34.

[3] Hao Dong and Jiangjun Zhong. 2024. Extended Equation Learner Solves Symbolic Regression Tasks with Evolutionary Components. *IEEE Transactions on Evolutionary Computation* (2024).

[4] Hao Dong and Jiangjun Zhong. 2025. A Survey on Symbolic Regression. *arXiv preprint arXiv:2211.10873* (2025).

[5] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. 2019. Neural Architecture Search: A Survey. *Journal of Machine Learning Research* 20, 55 (2019), 1–21.

[6] Sinan Ipek. 2026. Evaluating the Exp-Minus-Log Sheffer Operator for Battery Characterization. *arXiv preprint arXiv:2604.13873* (2026).

[7] Sinan Ipek. 2026. Hardware-Efficient Neuro-Symbolic Networks with the Exp-Minus-Log Operator. *arXiv preprint arXiv:2604.13871* (2026).

[8] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. *Proceedings of the International Conference on Learning Representations (ICLR)* (2015).

[9] Mikel Landajuela, Chak Shing Lee, Jiachen Yang, Ruben Glatt, Claudio P. Santiago, Ignacio Aravena, Terrell Mundhenk, Garrett Mulcahy, and Brenden K. Petersen. 2022. A Unified Framework for Deep Symbolic Regression. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35.

[10] Wenqiang Li, Weijun Li, Lina Yu, Min Wu, Linjun Sun, Jingyi Liu, Yanjie Li, and Songsong Tian. 2025. MetaSymNet: A Dynamic Symbolic Regression Network Capable of Evolving into Arbitrary Formulas. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[11] Nour Makke and Sanjay Chawla. 2024. Interpretable Scientific Discovery with Symbolic Regression: A Review. *Artificial Intelligence Review* 57, 2 (2024).

[12] Andrzej Odrzywolek. 2026. All Elementary Functions from a Single Operator. *arXiv preprint arXiv:2603.21852v2* (2026). Institute of Theoretical Physics, Jagiellonian University.

[13] Brenden K. Petersen, Mikel Landajuela, T. Nathan Mundhenk, Claudio P. Santiago, Soo K. Kim, and Joanne T. Kim. 2021. Deep Symbolic Regression: Recovering Mathematical Expressions from Data via Risk-Seeking Policy Gradients. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[14] Subham Sahoo, Christoph Lampert, and Georg Martius. 2018. Learning Equations for Extrapolation and Control. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 4442–4450.